Counting Distinct Elements in a Data Stream

Ziv Bar-Yossef[1⋆], T.S. Jayram[2], Ravi Kumar[2], D. Sivakumar[2], and Luca Trevisan[3⋆⋆]

[1] Computer Science Division, Univ. of California at Berkeley, Berkeley, CA 94720. zivi@cs.berkeley.edu

[2] IBM Almaden Research Center, 650 Harry Road, San Jose, CA 95120. {jayram, ravi, siva}@almaden.ibm.com

[3] Computer Science Division, Univ. of California at Berkeley, Berkeley, CA 94720. luca@cs.berkeley.edu

Abstract. We present three algorithms to count the number of distinct elements in a data stream to within a factor of $1 \pm \epsilon$. Our algorithms improve upon known algorithms for this problem, and offer a spectrum of time/space tradeoffs.

1 Introduction

Let $\mathbf{a} = a_1, \ldots, a_n$ be a sequence of n elements from the domain $[m] = \{1, \ldots, m\}$. The *zeroth-frequency moment* of this sequence is the number of distinct elements that occur in the sequence and is denoted $F_0 = F_0(\mathbf{a})$. In this paper we present three space- and time-efficient algorithms for approximating F_0 in the data stream model.

In the data stream model, an algorithm is considered efficient if it makes one (or a small number of) passes over the input sequence, uses very little space, and processes each element of the input very quickly. In our context, a data stream algorithm to approximate F_0 is considered efficient if it uses only $\mathrm{poly}(1/\epsilon, \log n, \log m)$ bits of memory, where $1 \pm \epsilon$ is the factor within which F_0 must be approximated.

Let $\epsilon, \delta > 0$ be given. An algorithm $\mathcal{A}$ is said to (ϵ, δ)-*approximate* F_0 if for any sequence $\mathbf{a} = a_1, \ldots, a_n$, with each $a_i \in [m]$, it outputs a number $\tilde{F}_0$ such that $\Pr[\left|F_0 - \tilde{F}_0\right| \leq \epsilon F_0] \geq 1 - \delta$, where the probability is taken over the internal coin tosses of $\mathcal{A}$. Two main parameters of $\mathcal{A}$ are of interest: the workspace and the time to process each item. We study these quantities as functions of the domain size m, the number n of elements in the stream, the approximation parameter ϵ, and the confidence parameter δ.

There are several reasons for designing algorithms for F_0 in the data stream model. Counting the number of distinct elements in a (column of a relational) table of data is a fairly fundamental problem in databases. This has applications

⋆ Part of this work was done while the author was visiting IBM Almaden Research Center. Supported by NSF Grant CCR-9820897.

⋆⋆ Work supported by a Sloan Research Fellowship and an NSF Career Award.

J.D.P. Rolim and S. Vadhan (Eds.): RANDOM 2002, LNCS 2483, pp. 1–10, 2002.
© Springer-Verlag Berlin Heidelberg 2002

to estimating the selectivity of queries, designing good plans for executing a query, etc.—see, for instance, [WVT90,HNSS96]. Another application of counting distinct elements is in routing of Internet traffic. The router usually has very limited memory, but it is desirable to have the router gather various statistical properties (say, the number of distinct destination addresses) of the traffic flow. The number of distinct elements is also a natural quantity of interest in several large data set applications (eg., the number of distinct queries made to a search engine over a week).

Flajolet and Martin [FM85] designed the first algorithm for approximating F_0 in the data stream (or what was then thought of as a one-pass) model. Unfortunately, their algorithm assumed the existence of hash functions with some ideal properties; it is not known how to construct such functions with limited space. Alon, Matias, and Szegedy [AMS99] built on these ideas, but used random pairwise independent hash functions [CW77,WC79] and gave an (ϵ, δ)-approximation algorithm for $\epsilon > 1$; their algorithm uses $O(\log m)$ space. For arbitrarily small ϵ, Gibbons and Tirthapura [GT01] gave an algorithm that used $S = O(1/\epsilon^2 \cdot \log m)$ space and $O(S)$ processing time per element; Bar-Yossef *et al.* [BKS02] gave an algorithm that used $O(1/\epsilon^3 \cdot \log m)$ space (and time per element) but that had some other nice property required for their application. Cohen [Coh97] considered this problem in the context of graph-theoretic applications; her algorithm is similar in spirit to that of [FM85,AMS99]; specifically, it has a high-level viewpoint similar to the first algorithm in this paper. However, the implementation is very different, and does not yield a $o(m)$ space algorithm.

One of the drawbacks of the algorithms of [GT01,BKS02] is that the space and time are the *product* of $\text{poly}(1/\epsilon)$ and $\log m$. Even with modestly small constants in the O notation and $\epsilon = 0.01$, the space required might be prohibitive in certain applications (eg., a router with very little memory or a database application where the frequency estimation is required to be piggy-backed on some other computation). This leads to the question of whether it is possible to obtain space/time upper bounds that are $\text{poly}(1/\epsilon) + \log m$. In this paper we achieve this, modulo factors of the form $\log \log m$ and $\log(1/\epsilon)$.

Results. We give three algorithms with different space-time tradeoffs for approximating F_0. Each of our algorithms is an improvement over any of the existing algorithms in either space or processing time or both.

We will state the bounds for our algorithms in terms of ϵ and $\log m$ (suppressing the dependence on n). This is without loss of generality: If indeed $m < n$, it is clearly advantageous to have an algorithm whose bounds depend on $\log m$ and not on $\log n$. If, on the other hand, $m > n$, we can employ a simple hashing trick (with $O(\log(m + n))$ space and time per element) that reduces the description of each stream element to $O(\log n)$ bits. Thus we will assume for the rest of the paper that $\log m = O(\log n)$. We will also assume that there exists $\epsilon_0 < 1$ such that the accuracy parameter ϵ given to the algorithms is at most ϵ_0. (Note that this is, in fact, the interesting case. We make this assumption explicit only so that we may abbreviate $\max\{1/\epsilon, \epsilon_0\}$ by $1/\epsilon$.)